SITEWATCH

Website & Service Monitoring

Linux Automation Project | Topic 4 | Report draft | 14 November 2026

Group information

Member 1: [enter full name and student ID]

Member 2: [enter full name and student ID]

Member 3: [enter full name and student ID]

Member 4: [enter if applicable]

1. Objective

Design a Linux/Bash monitoring utility for websites and host:port services. The system collects HTTP status and response time, tests DNS and TCP, stores historical checks, calculates observed uptime and notifies operators only when target status changes.

2. Project layout

The course-suggested layout is used, with ONE application shell script. Monitoring, state management, CSV storage, alerts and uptime reporting are all implemented inside bin/sitewatch.sh.

```
README.md user guide
bin/sitewatch.sh single main script
config/sitewatch.conf trusted configuration
systemd/sitewatch.service systemd oneshot unit
systemd/sitewatch.timer 3-minute timer
install.sh / uninstall.sh deployment/removal
docs/report.pdf this report draft
docs/screenshots/ screenshots of real tests
```

3. Design principles

Shell scripts use '#!/usr/bin/env bash' and 'set -euo pipefail'. The main script provides usage(), log(), debug(), check_deps(), main() and additional specialized functions. Configuration is sourced only from trusted files.

4. Monitoring process

1) Read URLS and SERVICES arrays, TIMEOUT, HTTP_MIN, HTTP_MAX and DATA_DIR from the configuration. Validate targets, ports, timeout values and dependencies.

2) Use 'getent ahosts' for DNS, 'nc -z -w' for TCP connectivity and 'curl -w' for HTTP code and total response time.

3) URL is UP only if DNS=OK, TCP=OPEN and final HTTP code is in the configured range (default 200-399). A TCP service is UP when DNS=OK and TCP=OPEN.

4) Append one timestamped check per target to results.csv. Remember last known target statuses in state.tsv. Log WARN ALERT on UP-to-DOWN and INFO RECOVERY on DOWN-to-UP, with no duplicate alert if unchanged.

5. Storage and uptime

results.csv columns: timestamp, type, target, dns, tcp, http_code, response_sec, status. state.tsv stores type, target and latest UP/DOWN state. sitewatch.log records timestamped INFO/WARN/ERROR messages.

Observed uptime percentage = (number of UP checks / number of total checks) * 100. This measures sampled checks rather than exact uninterrupted time.

6. CLI and errors

```
-h help -c FILE custom configuration
-v verbose logging -n dry run, no file writes
-r read uptime report from saved CSV, no new checks
```

Exit code 0 = successful check (even if a website is DOWN); 1 = dependency/config/runtime error; 2 = invalid argument. A missing config causes the oneshot systemd service to fail; a DOWN website is a valid monitoring result.

7. Automation and setup

Install with 'sudo bash install.sh'. The installer copies the single main script into /usr/local/lib/sitewatch/bin, config into /etc/sitewatch, and systemd units into /etc/systemd/system; re-installation preserves existing config.

The service uses Type=oneshot. The timer starts about one minute after boot and then every three minutes after the previous run becomes inactive (OnUnitInactiveSec=3min). Uninstall using 'sudo bash uninstall.sh'; --keep-data preserves results.

8. Validation checklist

[ ] 'shellcheck bin/sitewatch.sh install.sh uninstall.sh' passes.

[ ] 'bash bin/sitewatch.sh -h' displays usage, and '-n' shows configured targets without writing files.

[ ] A local Python HTTP server on 127.0.0.1:8000 reports UP with HTTP 200.

[ ] Stopping it reports DOWN and emits a WARN alert once per changed target; repeated DOWN emits no duplicate alert.

[ ] Restarting the server reports UP and emits a recovery message.

[ ] DNS failure and closed port tests yield DOWN without crashing.

[ ] CSV, state.tsv, logfile and '-r' uptime calculations are correct.

[ ] A fresh supported-OS VM install, 3-minute timer, journal and clean uninstall are demonstrated.

9. Current verification notes

A previous multi-script version was tested with GitHub Actions and Ubuntu 26.04.1. The single-script refactor was tested in a local Bash environment with UP/DOWN/recovery, output files, sampled uptime and error codes. Final deployment evidence must be repeated on a course-approved VM.

10. Course constraints and submission TODO

The course specifies Ubuntu Server 22.04/24.04 or Rocky Linux 9; Ubuntu 26.04.1 is not listed. This PDF is a DRAFT, not an assertion that all required evidence has already been captured.

Before submitting, replace the member placeholders with actual full names and IDs; insert real terminal screenshots under docs/screenshots/; update the report with results obtained on the VM used for the presentation.

Draft prepared for group review. Do not claim final OS/compliance verification without testing.